

Les Classes et la Programmation Orientée Objet (POO)

Objectif

L'objectif de cet atelier est d'abandonner les tables de hachage et les objets personnalisés "mous" ([PSCustomObject]) pour concevoir une architecture de données stricte. Vous allez modéliser une infrastructure sous forme de Classes PowerShell, intégrant des règles de validation, des méthodes et de l'héritage.

Exercice 1 : Énumérations et Classe de Base

Pour éviter qu'un administrateur n'utilise du texte libre pour définir l'état d'un serveur ("OK", "En panne", "Up"), vous allez figer ces états.

1. Création de l'énumération :

Déclarez une énumération (enum) nommée EtatMachine.

Elle doit contenir trois valeurs strictes : Inconnu, EnLigne, et Maintenance.

2. Création de la classe principale :

Déclarez une classe nommée Serveur.

Ajoutez-y les propriétés fortement typées suivantes :

- Nom (Chaîne de caractères)
- AdressIP (Chaîne de caractères)
- RAM_Go (Entier)
- Statut (De type EtatMachine, initialisé par défaut à [EtatMachine]::Inconnu)

3. Validation :

À l'extérieur de votre classe, tentez de l'instancier avec la méthode statique :

```
$MonServeur = [Serveur]::new()
```

Vérifiez le contenu de la variable \$MonServeur pour confirmer que la propriété Statut vaut bien Inconnu.

Exercice 2 : Verrouillage via le Constructeur

Actuellement, il est possible de créer un serveur "vide". Vous allez imposer des paramètres obligatoires lors de l'instanciation.

1. Ajout du constructeur paramétré :

À l'intérieur de la classe Serveur, ajoutez une méthode portant exactement le même nom que la classe (Serveur).

Cette méthode doit accepter 3 paramètres obligatoires : \$nom, \$ip et \$ram.

2. Logique interne (Le mot-clé \$this) :

Dans le constructeur, affectez les valeurs reçues aux propriétés de l'objet via \$this.Nom, \$this.AdressIP et \$this.RAM_Go.

3. **Protection contre les valeurs absurdes :**

Toujours dans le constructeur, ajoutez une condition : si la \$ram fournie est strictement inférieure à 2, déclenchez une erreur bloquante (throw "La RAM minimum requise est de 2 Go.").

4. **Validation :**

- Tentez de créer un serveur avec 1 Go de RAM : [Serveur]::new("SRV-TEST", "10.0.0.1", 1). Constatez le rejet immédiat.
- Créez un serveur valide avec 8 Go de RAM et vérifiez ses propriétés.

Exercice 3 : Intégration de Méthodes Actives

Au lieu d'utiliser des commandes externes pour manipuler ce serveur, vous allez lui apprendre à se gérer lui-même.

1. **Méthode d'action :**

Ajoutez une méthode nommée BasculerEnMaintenance() à la classe Serveur.

Règle d'or : Elle ne doit rien renvoyer ([void]).

Action : Elle doit simplement basculer la propriété Statut de l'objet sur [EtatMachine]::Maintenance.

2. **Méthode de restitution :**

Ajoutez une méthode nommée ObtenirBilan() qui renvoie obligatoirement une chaîne de caractères ([string]).

Elle doit construire et retourner (via le mot-clé return) la phrase suivante :

"Le serveur <Nom> (<AdresseIP>) dispose de <RAM_Go> Go. État actuel : <Statut>."

3. **Validation :**

Sur le serveur valide créé à l'exercice 2, appelez :

- \$MonServeur.BasculerEnMaintenance()
- \$MonServeur.ObtenirBilan()

Vérifiez que le bilan reflète bien le changement d'état.

Exercice 4 (Avancé) : L'Héritage (Spécialisation)

Vous allez créer une déclinaison de cette classe générique pour un type de serveur spécifique (un serveur web).

1. **Création de la classe fille :**

Déclarez une nouvelle classe nommée ServeurWeb qui hérite de Serveur.

Syntaxe : class ServeurWeb : Serveur { ... }

2. **Nouvelle propriété :**

Ajoutez-lui la propriété MoteurHTTP de type chaîne de caractères (ex: "Nginx", "IIS").

3. **Constructeur spécialisé (Le mot-clé base) :**

Créez un constructeur pour ServeurWeb acceptant 4 paramètres : \$nom, \$ip, \$ram et \$moteur.

Ce constructeur doit déléguer l'affectation du nom, de l'IP et de la RAM au constructeur parent en appelant : base(\$nom, \$ip, \$ram).

Il ne s'occupera que de faire `$this.MoteurHTTP = $moteur`.

4. **Validation finale :**

Instanciez un `ServeurWeb` :

```
$MonWeb = [ServeurWeb]::new("SRV-NGINX-01", "192.168.1.50", 4, "Nginx")
```

Appelez la méthode `$MonWeb.ObtenirBilan()`. Constatez que bien qu'elle n'ait pas été réécrite dans `ServeurWeb`, elle a été parfaitement héritée de la classe parente !